Άσκηση 2 HY-150

17/03/2025 - 30/03/2025

Σε αυτή την άσκηση θα φτιάξετε ένα πρόγραμμα που θα παράγει εικόνες με τη μορφή PPM που θα απεικονίζουν απλές σκηνές αποτελούμενες από τρίγωνα με την μέθοδο Ray Tracing.

Μπορείτε να διαβάσετε περισσότερα για τη μορφή εικόνας PPM και για το Ray Tracing εδω:
ttps://elearn.uoc.gr/pluginfile.php/338416/mod_resource/content/1/HY150-RayTracingInOneWeekend.pdf

A. Triangle class [20%]

Υλοποιήστε την κλάση Triangle η οποια αναπαριστά ένα τρίγωνο.
Η κλάση θα έχει τις εξής μεταβλητές:

- ***vec3 v0***, η πρώτη κορυφή του τριγώνου,
- ***vec3 v1***, η δεύτερη κορυφή του τριγώνου,
- ***vec3 v2***, η τρίτη κορυφή του τριγώνου,
- ***vec3 color***, το χρώμα του τριγώνου

Μπορείτε να προσθέσετε όποια άλλη μεταβλητή στην κλάση θεωρείται απαραίτητη και θα σας βοηθήσει στην υλοποίηση σας.

Η κλάση θα θα πρέπει να έχει την εξής μέθοδο:

- ***bool Hit(const ray&,)***, παίρνει ως παραμέτρους μια ακτίνα τύπου ray. Σε αυτήν την μέθοδο θα πρέπει να υλοποιήσετε διασταύρωση ακτίνας εναντίων τριγώνου έτσι ώστε να βρείτε αν η ακτίνα χτυπάει το τρίγωνο. Η μέθοδος επιστρέφει **true** αν υπάρχει χτύπημα μεταξύ της ακτίνας και του τριγώνου αλλιώς επιστρέφει **false**.

Υλοποιήστε setters και getters για τις 3 μεταβλητές της κλάσης αυτής.
Υλοποιήστε έναν constructor που θα παίρνει ως παραμέτρους *vec3 v0, vec3 v1, vec3 v2* και *vec3 color*.
(TIP: Συμβουλευτείτε το αρχείο Example.cpp για περισσότερες λεπτομέρειες για το πως μπορείτε να υπολογίσετε εαν μια ακτίνα χτυπάει ένα τρίγωνο)

B. Camera class [20%]

Υλοποιήστε την κλάση Camera η οποία αναπαριστά την κάμερα από την οποία θα βλέπουμε τον κόσμο μας.
Πιο συγκεκριμένα η κλάση θα έχει τις εξής μεταβλητές:

- ***vec3 center***, το κέντρο της κάμερας (η θέση της στο χώρο),
- ***int width***, το πλάτος της εικόνας που θα παραχθεί,
- ***float aspectW***, το πλάτος του aspect ratio, στην περίπτωση του 16/9, το 16
- ***float aspectH***, το ύψος του aspect ratio, στην περίπτωση του 16/9, το 9

Μπορείτε να προσθέσετε όποια άλλη μεταβλητή στην κλάση θεωρείται απαραίτητη και θα σας βοηθήσει στην υλοποίηση σας.

Η κλάση Camera θα πρέπει να περιέχει και τις εξής μεθόδους:

- ***vec3 GetPixelCenter(int, int)***, παίρνει ως παράμετρο τις συντεταγμένες ενός pixel και επιστρέφει την θέση του στον χώρο *(TIP: συμβουλευτείτε το αρχείο Example.cpp για περισσότερες πληροφορίες για την υλοποίηση)*

Υλοποιήστε ***getters*** για ***center, width και height***.

Υλοποιήστε έναν constructor που θα παίρνει ως παραμέτρους το *vec3 center* και *int width*, *float aspectW, float aspectH* και αρχικοποιεί την καμερα. *(TIP: Συμβουλευτείτε το αρχείο Example.cpp για περισσότερες λεπτομέρειες για την αρχικοποίηση της κάμερας)*

C. World class [20%]

Υλοποιήστε την κλάση World η οποία περιέχει όλα τα Triangles του κόσμου σε ένα vector.
Πιο συγκεκριμένα η κλάση θα έχει τις εξής μεταβλητές:

- ***vector<Triangle> Triangles***, ένα vector από Triangle,
- ***vec3 SkyColor***, το χρώμα του ουρανού,

Μπορείτε να προσθέσετε όποια άλλη μεταβλητή στην κλάση θεωρείται απαραίτητη και θα σας βοηθήσει στην υλοποίηση σας.

Η κλάση World θα πρέπει να περιέχει και τις εξής μεθόδους:

- ***void Add(Triange&)***, η οποία θα προσθέτει ένα νέο τρίγωνο στο Triangles vector.
- ***void Render(Camera&)***, παίρνει ως παράμετρο την κάμερα από την οποία θα "απαθανατίσει" τον κόσμο. Πιο συγκεκριμένα, διατρέχει όλα τα pixels της εικόνας σε μια διπλή λούπα και κάθε φορά δημιουργεί ένα νέο ray με origin το κέντρο της κάμερας και κατεύθυνση από το κέντρο της κάμερας προς την θέση του εκάστοτε pixel στο χώρο και τσεκάρει αν το ray χτυπάει κάποια από τα τρίγωνα του κόσμου. Αν χτυπάει κάποιο τρίγωνο, αποθηκεύει στο αρχείο το χρώμα του τριγώνου που χτύπησε, αλλιώς αποθηκεύει το sky color εφόσον δεν υπήρξε κάποιο χτύπημα. Αποθηκεύει κάθε φορά σε αρχείο το χρώμα σύμφωνα με το PPM format.

Υλοποιήστε έναν constructor που θα παίρνει ως παράμετρο το *vec3 sky_color* και ενα std::string με το όνομα του αρχείου που θα αποθηκευτεί η εικόνα. Δημιουργήστε έναν κόσμο που να εμπεριέχει τρια τρίγωνα.

D. Multiple files [10%]

Υλοποιήστε τις κλάσεις Triangles, Camera, World σε ξεχωριστά αρχεία .cpp. Οι δηλώσεις τους μπορεί να είναι σε ένα συγκεντρωτικό .h ή σε ξεχωριστά .h, αλλά η υλοποίηση τους πρέπει να είναι σε ξεχωριστά αρχεία .cpp.

E. Input and Output [30%]

Επεκτείνετε το πρόγραμμα σας διαβάζοντας από ένα αρχεόο "*world.txt*" τις προδιαγραφές του κόσμου που θα δημιουργήσετε χρησιμοποιώντας C++ input streams. Το αρχείο αυτό θα πρέπει να έχει τις εξής πληροφορίες, μια σε κάθε γραμμή:

- Το όνομα του αρχείου που θα αποθηκεύσετε την εικόνα σε μορφή PPM,
- Το κέντρο της κάμερας,
- Το πλάτος της εικόνας,
- Το πλάτος του aspect ratio,
- Το ύψος του aspect ratio,

- Το χρώμα του ουρανού,
- Το πλήθος των τριγώνων,
- Το χρώμα, τις θέσεις των τριών κορυφών του κάθε τριγώνου (σε νέα γραμμή κάθε φορά για το κάθε διαφορετικό τρίγωνο).

Παρακάτω φαίνεται ένα παράδειγμα ενος τετοιου αρχειου:

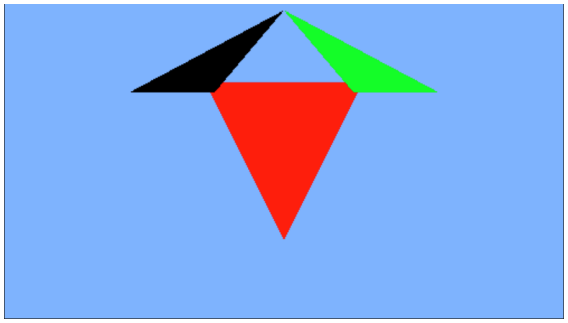
```
my_rt_image
0.0 0.0 0.0
500
16 9
0.5 0.7 1.0
3
1 0 0 -0.5 0.5 -1.0 0.5 0.5 -1.0 0 -0.5 -1.0
0 1 0 0 0.96 -1 0.44 0.44 -1 0.98 0.44 -1
0 0 0 0 0.96 -1 -0.44 0.44 -1 -0.98 0.44 -1
```

Χρησιμοποιήστε τις παραμέτρους που διαβάσατε από το αρχείο και παράξτε μια εικόνα με τη μορφή PPM που θα απεικονίζει τον κόσμο με την μέθοδο Ray Tracing χρησιμοποιώντας τις κλάσεις που έχετε φτιάξει έως τώρα.

Υλοποιήστε μια παραλλαγή της `write_color` σας έχει δοθεί έτσι ώστε να κάνετε output σε ένα αρχείο, με όνομα αυτό που σας έδωσε ο χρήστης κάνοντας χρήση C++ output streams, το χρώμα του κάθε pixel.

Είναι υποχρεωτικό να γίνει η χρήση Exceptions για τον έλεγχο των input / output streams και να εμφανίζονται τα κατάλληλα μηνύματα.

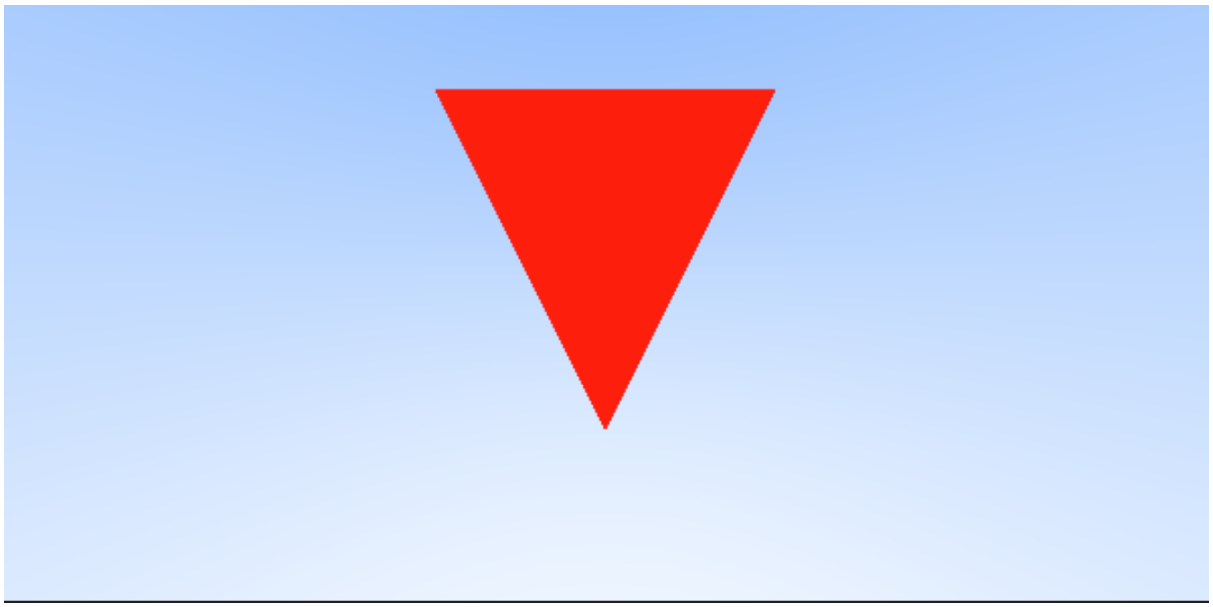
Η εικόνα που δημιουργήθηκε χρησιμοποιώντας τις τιμές του παραπάνω αρχείου είναι η εξής:



Υλοποίηση

Για την υλοποίηση θα χρειαστεί να κατεβάσετε τον φάκελο με το υλικό της άσκησης 2. Αυτός ο φάκελος περιέχει διάφορες βιβλιοθήκες με συναρτήσεις που θα σας βοηθήσουν, καθώς και ένα παράδειγμα με τη χρήση αυτών για τη δημιουργία ενος απλου τριγωνου με την μεθοδο Ray Tracing.

Example.cpp
Αυτό το αρχείο περιέχει κώδικα για τη δημιουργία της παρακάτω εικόνας οπως και για τις ρυθμισεις και σεταρισμα της καμερας:



Οι πρώτες γραμμές στον κώδικα:

```
#include "vec3.h"
#include "color.h"
#include "ray.h"
```

χρειάζονται για να υπάρχει πρόσβαση σε βοηθητικές συναρτήσεις όπως η `write_color(...)`. Αυτές θα χρειαστεί να τις συμπεριλάβετε και στο αρχείο του κώδικά σας αν θέλετε να χρησιμοποιήσετε αυτές τις συναρτήσεις.

vec3.h
Αυτό το αρχείο περιέχει βοηθητικές συναρτήσεις για διανύσματα και πράξεις με αυτά. Ο τρόπος χρήσης των συναρτήσεων αυτών υπάρχει στο αρχείο Example.cpp.

color.h
Αυτό το αρχείο περιέχει βοηθητικές συναρτήσεις για τα χρώματα(color). Παραδείγματα χρήσης αυτών υπάρχουν στο αρχείο Example.cpp.

ray.h
Αυτό το αρχείο περιέχει βοηθητικές συναρτήσεις για ακτινες(rays) και πράξεις με αυτές. Ο τρόπος χρήσης των συναρτήσεων αυτών υπάρχει στο αρχείο Example.cpp.

Assignment2.cpp
Εδώ θα χρειαστεί να γράψετε τον κώδικά σας.
Δομήστε σωστά τον κώδικά σας και σπάστε τις διάφορες λειτουργίες σε κατάλληλες κλασεις και αρχεια που σας ζητηθηκαν.

Δοκιμή του Προγράμματος σας

Για να εμφανίσετε τις εικόνες PPM μπορείτε να:

- εγκαταστήσετε το plugin PBM/PPM/PGM Viewer for Visual Studio Code για το Visual Studio Code ή
- να ανατρέξετε εδώ https://jumpshare.com/viewer/ppm ή
- εδώ https://0xc0de.fr/webppm/

Υποβολή

- Η άσκηση είναι καθαρά ατομική.
- Τα παραδοτέα αρχεία θα ελεγχθούν για αντιγραφές. Τυχόν αντιγραφή ισοδυναμεί με μηδενισμό σας.

Παραδώστε όλο τον φάκελο με την υλοποίηση σας σε ένα .zip αρχείο με το όνομα Assignment2_csdXXXX.zip.